



PROJET DE CALCUL SCIENTIFIQUE

2023-2024

Méthodes d'itération de sous-espaces et applications à la compression d'image

Ragot Cyrian

Cebula Baptiste

Table des matières

1	Introduction	3
2	Limitations de la méthode de la puissance itérée	3
3	Extension de la méthode de la puissance itérée au calcul d'espaces propres dominants	4
3.1	Premier algorithme simple	4
3.2	Amélioration en utilisant la projection de Raleigh-Ritz	5
3.3	Amélioration avec une approche par blocs	5
3.4	Amélioration par déflation	6
4	Applications numériques	7
4.1	Différences entre les 4 types de matrices	7
4.1.1	Type 1	7
4.1.2	Type 2	7
4.1.3	Type 3	7
4.1.4	Type 4	7
4.2	Performances des algorithmes pour différentes matrices	8
5	Application à la compression d'images	10
5.1	Première analyse d'une image à compresser	10
5.2	Résultats obtenus	10
6	Conclusion	12

1 Introduction

Ce document a pour objectif de présenter des méthodes itératives de calcul de couples propres ainsi que de comparer l'efficacité des algorithmes associés. Dans un second temps nous appliquerons ces algorithmes à la compression d'images.

2 Limitations de la méthode de la puissance itérée

Dans cette partie nous utilisons la méthode de la puissance itérée avec déflation pour calculer les couples propres d'une matrice. Le premier algorithme utilisé est implanté dans le fichier **power.v11**.

Algorithm 1 Puissance itérée

Input: Matrice $A \in \mathbb{R}$
Output: (λ_1, v_1) couple propre dominant
 $v \in \mathbb{R}^n$
 $z = A \cdot v$
 $\beta = v^T \cdot z$
while $|\beta \cdot v - z|/|\beta| < \epsilon$ **do**
 $y \leftarrow A \cdot v$
 $v \leftarrow y/|y|$
 $z \leftarrow A \cdot v$
 $\beta \leftarrow v^T \cdot z$
end while
 $\lambda_1 \leftarrow \beta$ and $v_1 \leftarrow v$

Cependant, nous pouvons apporter une première amélioration à cet algorithme en remarquant que l'on effectue deux fois le produit $A \cdot v$. L'algorithme suivant, implanté dans le fichier **power.v12**, permet de passer à un seul calcul du produit $A \cdot v$ et donc de sauver n multiplication à chaque itération.

Algorithm 2 Puissance itérée améliorée

Input: Matrice $A \in \mathbb{R}$
Output: (λ_1, v_1) couple propre dominant
 $v \in \mathbb{R}^n$
 $z = A \cdot v$
 $\beta = v^T \cdot z$
while $|\beta \cdot v - z|/|\beta| < \epsilon$ **do**
 $v \leftarrow z/|z|$
 $z \leftarrow A \cdot v$
 $\beta \leftarrow v^T \cdot z$
end while
 $\lambda_1 \leftarrow \beta$ and $v_1 \leftarrow v$

Nous pouvons comparer l'efficacité de ces algorithmes et de l'algorithme implanté par matlab **eig** avec des matrices de tailles différentes. Les résultats obtenus pour des matrices de type 1 et 2 sont présentés dans les tableaux suivant.

Algorithme	100×100	200×200	500×500
eig	0.01s	0.01s	0.08s
power_v11	0.34s	1.4s	14.9s
power_v12	0.28s	0.29s	8.37s

TABLE 1 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 1

Algorithme	100×100	200×200	500×500
eig	0.0s	0.01s	0.06s
power_v11	0.05s	0.04s	0.25s
power_v12	0.03s	0.03s	0.17s

TABLE 2 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 2

La comparaison entre les algorithmes est similaire pour des matrices de type 3 et 4. Pour des matrices assez grandes on remarque bien la différence d'efficacité entre l'algorithme 1 et l'algorithme 2, en divisant par deux le temps d'exécution. Cependant, cet algorithme n'est pas le plus efficace, en effet l'algorithme **eig** est bien plus performant. La lenteur de la méthode de la puissance itérée avec déflation peut être expliquée par le critère de précision de sortie de boucle qui prolonge le temps de convergence. De plus, l'opération de déflation après chaque calcul de couple propre ajoute des opérations et le choix arbitraire de $v \in \mathbb{R}^n$ à l'initialisation peut être trop éloigné de la solution recherchée.

3 Extension de la méthode de la puissance itérée au calcul d'espaces propres dominants

Notre objectif est maintenant de calculer un espace propre dominant directement en évitant de calculer tout le spectre de la matrice d'entrée A . Nous allons pour cela adapter l'algorithme de la puissance itérée vu précédemment et nous allons tenter plusieurs améliorations au fur et à mesure de cette partie.

3.1 Premier algorithme simple

Ce premier algorithme simple est implanté dans **subspace_iter_v0**. Comme décrit dans le sujet du projet, il est nécessaire d'orthonormaliser les colonnes de la matrice résultat de chaque itération car sinon l'algorithme va converger vers une matrice dont les colonnes sont toutes le même vecteur dominant. Pour trouver les m premiers vecteurs dominants dans cette matrice à la fin des itérations il faut orthonormaliser à chaque itération.

Ensuite, le calcul du spectre complet de H n'est pas problématique car H est de taille $m \times m$ et est donc plus petite que A de taille $n \times n$, par rapport à l'algorithme de la partie 2 on gagne quand même $n - m$ de complexité.

3.2 Amélioration en utilisant la projection de Raleigh-Ritz

Dans le cadre d'une application à l'analyse en composantes principales, il faut pouvoir calculer la somme des valeurs propres dominantes à chaque itération pour la comparer à un pourcentage de la trace de la matrice A . Pour cela, le quotient de Rayleigh doit maintenant être calculé dans l'itération. C'est l'adaptation de Rayleigh-Ritz. L'algorithme est décrit en dessous et le code est fournis dans les fichiers **subspace_iter.v1** et **rayleigh_ritz_projection**. Commentons les étapes de l'algorithme en ajoutant les lignes de code correspondantes.

Algorithm 3 Raleigh-Ritz projection

Input: Matrix $A \in \mathbb{R}^{n \times n}$ and an orthonormal set of vectors V . ▷ Lignes 3 et 4
Output: The approximate eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .
▷ Lignes 7 et 8
 Compute the Rayleigh quotient $H = V^T \cdot A \cdot V$ ▷ Ligne 11
 Compute the spectral decomposition $X \cdot \Lambda_{out} \cdot X^T = H$, where the eigenvalues of H ($diag(\Lambda_{out})$) are arranged in descending order of magnitude. ▷ Lignes 13 et 14
 Compute $V_{out} = V \cdot X$. ▷ Ligne 16

Algorithm 4 Subspace iteration method v1 with Raleigh-Ritz projection

Input: Symmetric matrix $A \in \mathbb{R}^{n \times n}$, tolerance ε , *MaxIter* (max nb of iterations) and *PercentTrace* the target percentage of the trace of A ▷ Lignes 5 à 9
Output: n_{ev} dominant eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} . ▷ Lignes 12 à 14
 Generate an initial set of m orthonormal vectors $V \in \mathbb{R}^{n \times m}$; $k = 0$; *PercentReached* = 0 ▷ Lignes 48 à 49
repeat
 $k = k + 1$ ▷ Ligne 54
 Compute Y such that $Y = A \cdot V$ ▷ Ligne 56
 $V \leftarrow$ orthonormalisation of the columns of Y ▷ Ligne 58
 Rayleigh-Ritz projection applied on matrix A and orthonormal vectors V ▷ Ligne 61
 Convergence analysis step : save eigenpairs that have converged and update *PercentReached* ▷ Lignes 64 à 115
until (*PercentReached* > *PercentTrace* or $n_{ev} = m$ or $k > MaxIter$)

3.3 Amélioration avec une approche par blocs

L'objectif est maintenant d'améliorer notre algorithme **subspace_iter** pour diminuer le nombre de calculs effectués et donc augmenter les performances du programme en termes de temps. Une première manière de réaliser cette amélioration est d'adopter une approche par blocs.

L'objectif est de remplacer le produit $V = A \cdot V$ par un produit $V = A^p \cdot V$. Cela permet en un produit matriciel d'avoir autant d'effet du point de vue de la convergence de V que plusieurs itérations de la boucle et réduit par conséquent le nombre d'itérations

nécessaire pour que V converge (avec un rapport de p). Ceci est très efficace pour augmenter les performances de notre algorithme, car chaque itération de la boucle implique une orthonormalisation de la matrice qui est très coûteuse. De plus, une fois A^p calculée, les produits $A^p \cdot V$ et $A \cdot V$ sont tous deux d'une complexité en $O(m \cdot n^2)$. Ainsi, implémenter le produit $V = A^p \cdot V$ devrait théoriquement diviser le temps d'exécution du programme par environ p par rapport au produit $V = A \cdot V$.

Cependant, cette version de l'algorithme impose de calculer A^p . Ce calcul doit être fait de la manière la plus optimisée possible afin que le temps de calcul utilisé pour déterminer A^p ne soit pas équivalent ou plus important que le temps "gagné" grâce à l'approche par blocs. Supposons que A soit une matrice de taille $n \times n$. Le calcul $A^2 = A \cdot A$ a donc une complexité en $O(n^3)$. En supposant $p > 0$, le calcul $A^p = A \cdot A \cdot A \dots A$ a une complexité en $O((p-1) \cdot n^3)$. Nous avons donc cherché d'autres manières de calculer A^p avec une complexité moins importante. Voici l'algorithme que nous avons trouvé :

```
% on construit A^p en optimisant le nombre de produits matriciels
Amul = eye(size(A)); % Matrice dont la construction doit aboutir à A^p
A_puissance = A; % Matrice dont la valeur vaut A^(2^i) avec i le nombre d'itérations de la boucle while suivante
pMul = p; % Valeur décroissant permettant de déterminer l'avancement de la construction de la matrice A^p
while (pMul ~= 0)
    if mod(pMul,2) == 1
        Amul = Amul*A_puissance;
        pMul = pMul-1;
    end
    pMul = pMul/2;
    if pMul ~= 0
        A_puissance = A_puissance*A_puissance;
    end
end
```

FIGURE 1 – Algorithme de calcul optimisé de A^p

Cet algorithme diminue le nombre de produits matriciels réalisé par rapport au calcul $A \cdot A \cdot A \dots A$. $\forall p \geq 2$, la complexité de cet algorithme est minimum en $O(\lfloor \log_2(p) \rfloor + 1 \cdot n^3)$ (si p est de la forme $p = 2^k$, avec $k \in \llbracket 1; +\infty \rrbracket$). La complexité maximum est elle en $O((2\lfloor \log_2(p) \rfloor - 1) \cdot n^3)$ (si p est de la forme $p = 2^k - 1$, avec $k \in \llbracket 1; +\infty \rrbracket$). On s'aperçoit donc que les gains par rapport à l'autre méthode sont conséquents, en particulier lorsque la valeur de p est élevée. On note aussi qu'il est plus efficace avec cet algorithme de définir p de la forme $p = 2^k$, avec $k \in \llbracket 1; +\infty \rrbracket$.

Après avoir implémenté cette méthode, nous avons pu tester son efficacité en fonction de plusieurs valeurs de p . On s'aperçoit que comme prévu, les temps de calculs sont environ inversement proportionnels à p . Cependant, lorsque p devient trop important ($p > 64$ d'après ce que nous avons mesuré), la perte de précision impliquée par le calcul de A^p fait que le calcul ne converge plus.

3.4 Amélioration par déflation

Une autre possibilité d'amélioration de cet algorithme est l'utilisation de la méthode de la déflation. Cette méthode se base sur le fait que les colonnes de V convergent les unes après les autres dans l'ordre. Cela offre la possibilité de ne plus inclure les colonnes ayant déjà convergé dans les calculs, permettant par conséquent de diminuer la complexité de ceux-ci.

Dans les algorithmes **subspace_iter_v1** et **subspace_iter_v2** les colonnes de V ayant déjà convergé sont incluses dans les calculs essayant de faire converger les plus petites valeurs propres. Par conséquent, leur précision va augmenter. On peut supposer que le changement proposé dans l'algorithme **subspace_iter_v3**, en plus de diminuer les temps de calcul, va faire que les colonnes ayant convergé ne seront plus incluses dans d'autres calculs, menant au fait que leur précision ne va pas augmenter, et stagner légèrement en dessous de ϵ .

4 Applications numériques

4.1 Différences entre les 4 types de matrices

Pour tester nos algorithmes, nous utilisons différents types de matrices qui ont des valeurs propres réparties de manière différente. Pour comparer ces 4 types de matrices, nous utilisons des matrices de taille $n \times n = 500 \times 500$. Cela aura un impact sur les valeurs propres des matrices de type 1, 3 et 4.

Pour ces différents types de matrices, les valeurs propres sont plus ou moins conditionnées, les plus conditionnées étant celles du type 2. Les types les mieux distribués sont les types 1 et 4. De plus, les types 2, 3 et 4 possèdent des valeurs propres entre 0 et 1 tandis que le type 1 possède des grandes valeurs propres jusqu'à une valeur de 500.

4.1.1 Type 1

Conditionnement : $cond = n$
Valeurs propres : $\lambda_i = n - i + 1$

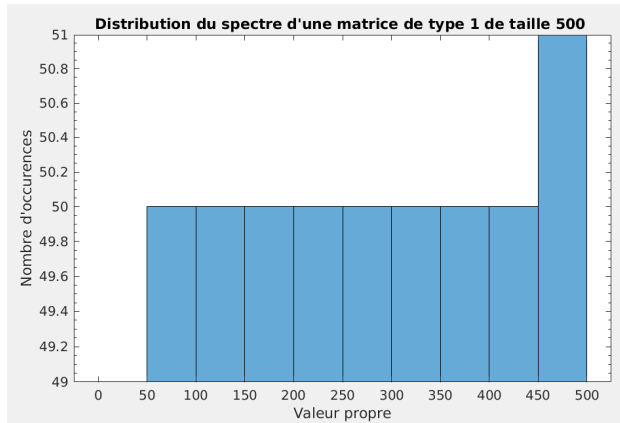


FIGURE 2 – Distribution type 1

4.1.2 Type 2

Conditionnement : $cond = 10^{10}$
Valeurs propres : $\lambda_i = e^{\log(1/cond) \times rand(1)}$

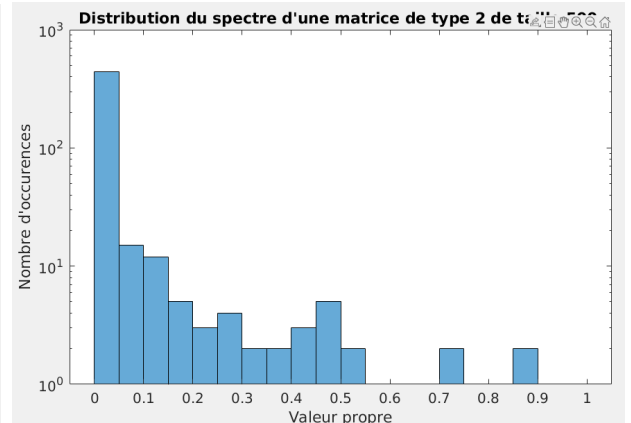


FIGURE 3 – Distribution type 2

4.1.3 Type 3

Conditionnement : $cond = 10^5$
Valeurs propres : $\lambda_i = (cond^{-1/n-1})^{i-1}$

4.1.4 Type 4

Conditionnement : $cond = 10^2$
Valeurs propres : $\lambda_i = (n-i) \times (1 - 1/cond) / (n-1) + 1/cond$

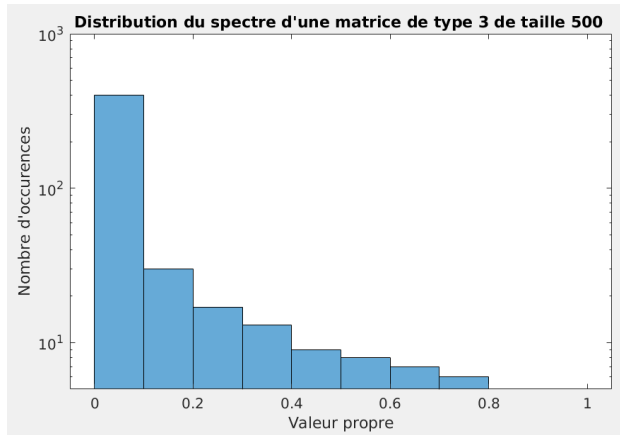


FIGURE 4 – Distribution type 3

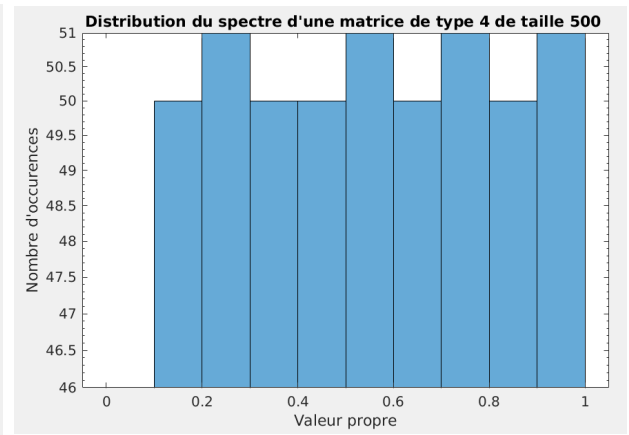


FIGURE 5 – Distribution type 4

4.2 Performances des algorithmes pour différentes matrices

Algorithme	100 × 100	200 × 200	300 × 300
eig	0.001s	0.001s	0.001s
power_v11	0.28s	1.09s	3.69s
power_v12	0.20s	0.67s	1.66s
subspace_iter_v0	0.48s	1.02s	1.87s
subspace_iter_v1	0.098s	0.27s	1.16s
subspace_iter_v2 ($p = 1$)	0.098s	0.33s	1.42s
subspace_iter_v2 ($p = 8$)	0.031s	0.063s	0.25s
subspace_iter_v2 ($p = 32$)	0.016s	0.016s	0.15s
subspace_iter_v3 ($p = 1$)	0.047s	0.27s	0.53s
subspace_iter_v3 ($p = 8$)	0.016s	0.047s	0.094s
subspace_iter_v3 ($p = 32$)	0.012s	0.012s	0.031s

TABLE 3 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 1

Algorithme	300 × 300	500 × 500	1000 × 1000
eig	0.001s	0.078s	0.17s
power_v11	0.15s	2.19s	2.28s
power_v12	0.10s	0.12s	1.20s
subspace_iter_v0	0.56s	0.58s	3.11s
subspace_iter_v1	0.031s	0.13s	0.77s
subspace_iter_v2 ($p = 1$)	0.012s	0.14s	0.64s
subspace_iter_v2 ($p = 8$)	0.031s	0.10s	0.25s
subspace_iter_v2 ($p = 32$)	0.062s	0.078s	0.44s
subspace_iter_v3 ($p = 1$)	0.062s	0.12s	0.26s
subspace_iter_v3 ($p = 8$)	0.016s	0.016s	0.20s
subspace_iter_v3 ($p = 32$)	0.007s	0.18s	0.27s

TABLE 4 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 2

Algorithme	300 × 300	500 × 500	1000 × 1000
eig	0.001s	0.031s	0.16s
power_v11	0.047s	0.43s	15.4s
power_v12	0.031s	0.30s	8.03s
subspace_iter_v0	0.39s	0.67s	2.69s
subspace_iter_v1	0.048s	0.047s	0.70s
subspace_iter_v2 ($p = 1$)	0.056s	0.093s	0.98s
subspace_iter_v2 ($p = 8$)	0.016s	0.047s	0.51s
subspace_iter_v2 ($p = 32$)	0.032s	0.062s	0.40s
subspace_iter_v3 ($p = 1$)	0.062s	0.12s	0.92s
subspace_iter_v3 ($p = 8$)	0.047s	0.031s	0.28s
subspace_iter_v3 ($p = 32$)	0.016s	0.055s	0.37s

TABLE 5 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 3

Algorithme	300 × 300	250 × 250	350 × 350
eig	0.001s	0.001s	0.001s
power_v11	0.81s	1.89s	6.34s
power_v12	0.25s	0.78s	4.31s
subspace_iter_v0	0.56s	1.76s	4.12s
subspace_iter_v1	0.20s	0.55s	2.51s
subspace_iter_v2 ($p = 1$)	0.048s	0.45s	2.98s
subspace_iter_v2 ($p = 8$)	0.031s	0.062s	0.37s
subspace_iter_v2 ($p = 32$)	0.016s	0.032s	0.094s
subspace_iter_v3 ($p = 1$)	0.10s	0.48s	2.31s
subspace_iter_v3 ($p = 8$)	0.016s	0.032s	0.30s
subspace_iter_v3 ($p = 32$)	0.001s	0.016s	0.062s

TABLE 6 – Comparaison du temps d'exécution des algorithmes pour des matrices de type 4

L'efficacité de nos algorithmes dépend donc beaucoup du type de matrice considéré et de sa taille. Les premiers algorithmes *power_v11* et *power_v12* sont généralement plus efficaces sur des petites matrices mais se font totalement surpasser par les algorithmes à sous-espaces pour des grandes matrices, ce qui était prévu.

L'algorithme *eig* est une bonne référence d'algorithme très efficace et on peut voir que l'on s'en rapproche avec les algorithmes *subspace_v2* et *subspace_v3*, souvent avec des valeurs de p grandes.

De manière générale, les différentes améliorations de *subspace_iter* fonctionnent mais dépendent beaucoup de la répartition des valeurs propres car *subspace_iter_v2* est parfois plus performant que *subspace_iter_v3*. De même, on remarque que augmenter p améliore les résultats généralement mais ce n'est pas acquis pour tous les types de matrices. Il faudrait pour chaque type, trouver la meilleure valeur de p .

5 Application à la compression d'images

Une image en noir et blanc est représentée par une matrice de taille $q \times p$. Lors de la compression par SVD, cette image est représentée par un triplet (Σ_k, U_k, V_k) dont les dimensions sont $k \times k$ pour Σ_k , $q \times k$ pour U_k et $p \times k$ pour V_k . Alors, lorsque $k < q < p$, on peut représenter cette image avec moins de valeurs.

5.1 Première analyse d'une image à compresser

Les algorithmes testés dans la partie précédente peuvent mieux fonctionner selon le type et la taille de la matrice considérée. Dans cette sous partie nous analysons donc la répartition des valeurs propres de la matrice $M = I \times I^T$, matrice à laquelle nous appliquons les algorithmes précédents pour la compression d'images (I représente l'image en niveau de gris). Voici la distribution du spectre de M pour l'image BD_Asterix_1.

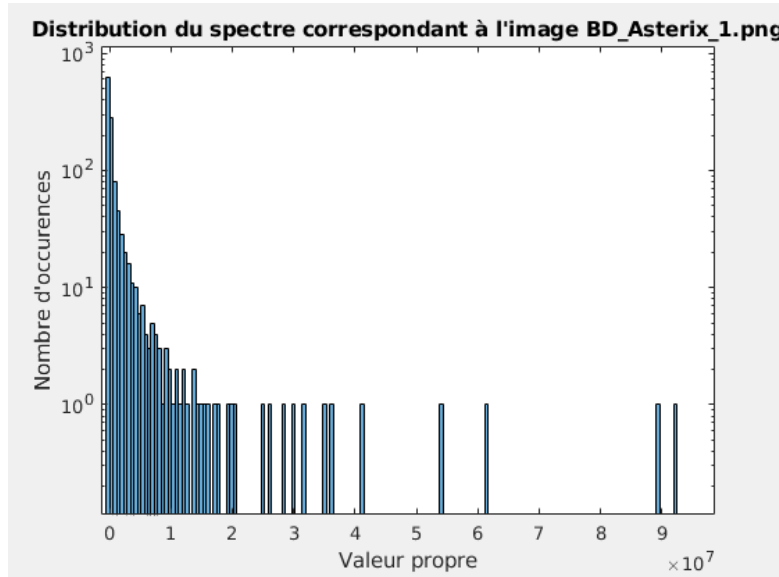
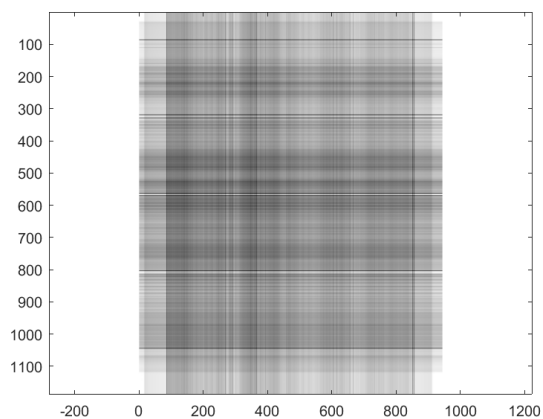
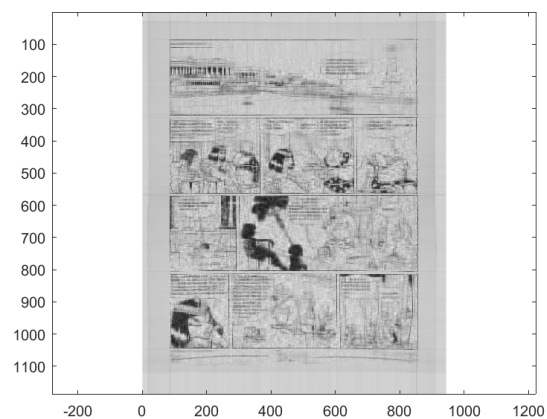
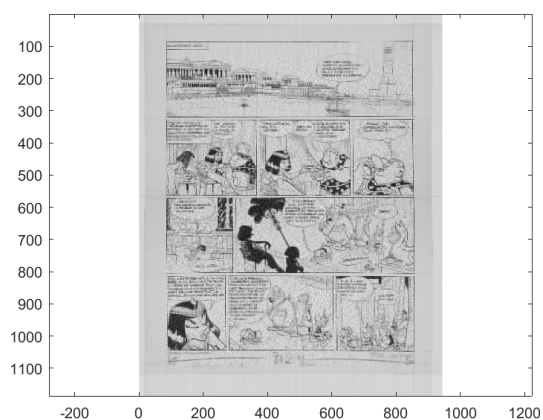
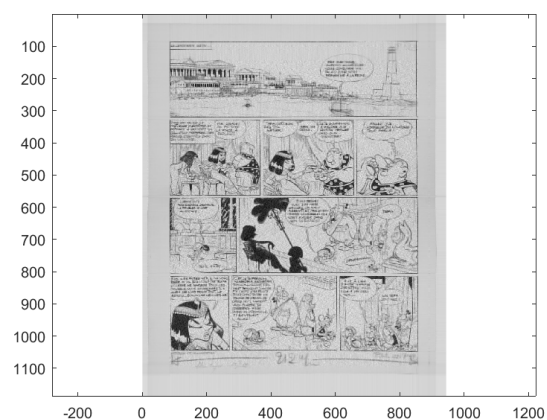
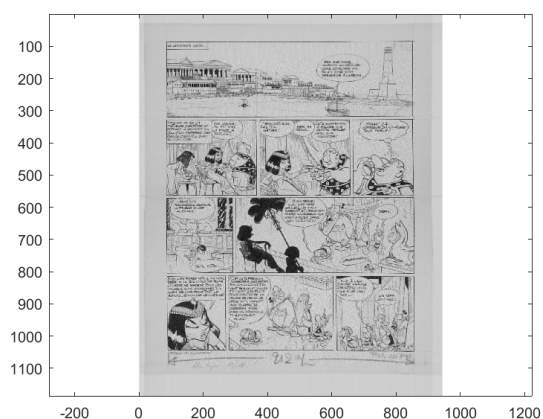
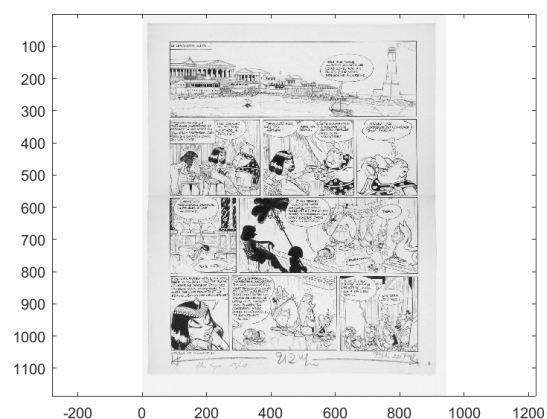


FIGURE 6 – Distribution du spectre pour l'image BD_Asterix_1

La matrice M semble donc être de type 2 ou 3 (voir figures partie 4.1) et sa taille est de 1187×1187 . On s'attend alors à obtenir une compression plus rapide pour les algorithmes *subspace_iter_v2* et *subspace_iter_v3* (voir résultats partie 4.2).

5.2 Résultats obtenus

On constate comme attendu que l'image devient de plus en plus nette au fur et à mesure que k augmente. On débute par une image illisible pour $k = 1$, puis on distingue de mieux en mieux les formes, on commence à pouvoir lire les dialogues vers $k = 150$ et on fini par avoir une image complètement nette pour $k = 500$. La courbe de $\|I - I_k\|$ correspond à ce que l'on peut constater visuellement et montre que ces deux matrices deviennent de plus en plus proches au fur et à mesure que k augmente. Cela nous montre ainsi que l'on peut choisir le niveau de compression, et donc par conséquent le niveau de détail, d'une image en la compressant via cette méthode, en changeant k .

FIGURE 7 – eig , $k = 1$ FIGURE 8 – eig , $k = 41$ FIGURE 9 – eig , $k = 81$ FIGURE 10 – eig , $k = 121$ FIGURE 11 – eig , $k = 161$ FIGURE 12 – eig , $k = 500$

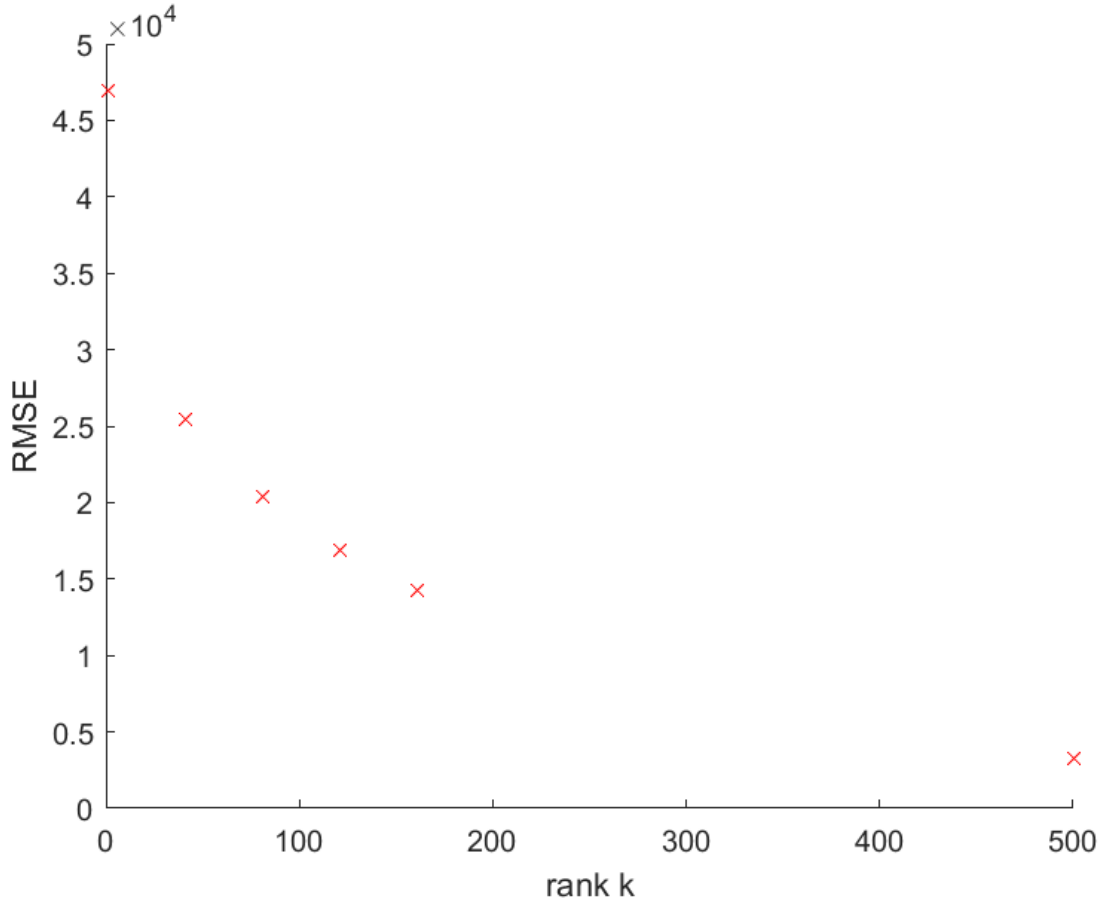


FIGURE 13 – Courbe représentant $\|I - I_k\|$ en fonction de k pour la fonction *eig*

On s'aperçoit également que la précision de la compression, ici représentée par $\|I - I_k\|$, ne varie pas selon l'algorithme utilisé. Cela montre que les algorithmes que nous avons écrits sont suffisamment précis pour ne pas perdre d'informations de par leur construction en elle-même.

Algorithme	$k = 100$	$k = 200$	$k = 500$
eig	18590	12112	3218
subspace_iter_v0	18590	12112	3218

TABLE 7 – Comparaison de $\|I - I_k\|$ selon les algorithmes et les valeurs de k

6 Conclusion

Finalement, nous avons pu observer des défauts dans l'algorithme de la puissance itérée, en particulier le fait qu'il calcule toutes les valeurs propres une à une. Ainsi, nous avons pu l'adapter au calcul d'espaces propres dominants qui est utile pour plusieurs applications comme le calcul de SVD dans la compression d'images. Plusieurs améliorations de cet algorithme ont été proposées et les résultats indiquent principalement une grande dépendance à la taille et au type des matrices considérées, c'est-à-dire à la répartition des valeurs propres.

Les résultats de la partie sur l'application à la compression d'images nous permettent de visualiser l'importance du choix de la taille du sous espace dominant (la valeur de k) sur la qualité de la compression, une valeur trop faible conduit à une mauvaise qualité tandis qu'une valeur trop grande allonge le temps de calcul et ne permet pas de compresser l'image aussi bien que souhaité.

Cependant, nous n'avons malheureusement pas réussi à traiter l'entièreté de la partie à la compression d'image.